

Searching in an array

Mostafa Faik

September 15, 2024

Introduction

Searching for a key element in an **unsorted array** is computationally expensive due to the need to check each element one by one. As array sizes grow, this process becomes increasingly inefficient. However, if the array is **sorted**, more efficient search methods become possible, significantly reducing the time needed to find an element. In this report, we explore three search methods: **unsorted search**, **sorted search**, and **binary search**. The **unsorted search** is an approach that sequentially checks each element in the array, while the **sorted search** leverages the fact that data is ordered to terminate the search early if the current element exceeds the key. Finally, we implement a more efficient **binary search** algorithm, which drastically improves performance.

Unsorted Search

The first task involved implementing a simple search algorithm for an **unsorted array**. In an **unsorted search**, the algorithm iterates through the array, comparing each element with the key. If the key is found, the search stops. If not, the algorithm continues until all elements have been checked.

Case	n	Time
1	100	34.72 ns
2	200	71.54 ns
3	400	157.86 ns
4	800	313.54 ns
5	1600	639.87 ns
6	3200	1323.16 ns

Table 1: Unsorted Search Time Measurements

We can observe that this method is simple but inefficient, as it requires checking each element in the worst-case scenario, leading to a time complex-

ity of $O(n)$. The results also confirm that the time taken by the unsorted search increases linearly with the size of the array, as expected. The time taken roughly doubles with each doubling of the array size. Searching an array of one million elements would be quite time-consuming due to this linear relationship, so for an array with one million elements, the search becomes highly inefficient for real world applications. But, based on our program and our assignment, it would take approximately 413 microseconds to search through one million elements.

Sorted Arrays

The second task involved optimizing the search algorithm for arrays that are already **sorted**. In the **sorted search**, we optimize the **unsorted search** by terminating the loop early if the current element in the array exceeds the key. This optimization takes advantage of the sorted order of the data.

Case	n	Time
1	100	31.56 ns
2	200	69.32 ns
3	400	150.48 ns
4	800	301.14 ns
5	1600	590.22 ns
6	3200	1232.88 ns

Table 2: Sorted Array Time Measurements

The measurements from the table above, shows that this minor optimization yields an improvement over the **unsorted search**, particularly in cases where the key is smaller than the larger elements in the array. While this optimization helps for certain key positions, the overall performance still follows an $O(n)$ complexity. However, for smaller key values in large arrays, this method can save time by exiting early. While this method is more efficient than **unsorted search**, it is still not suitable for large data sets when compared to more advanced methods. If the array contains one million elements, the search may need to check up to one million elements.

Binary Search

In the third task, we explored the **binary search** method, a more efficient search algorithm for **sorted arrays**. **Binary search** is a classic algorithm that exploits the sorted order of the array. By repeatedly halving the search space, the **binary search** quickly narrows down the possible location of the key, resulting in a significantly lower time complexity.

Case	n	Time
1	100	109.21 ns
2	200	126.04 ns
3	400	149.10 ns
4	800	161.40 ns
5	1600	179.77 ns
6	3200	198.44 ns

Table 3: Binary Search Time Measurements

We can observe that the **binary search** algorithm is highly efficient for large datasets due to its logarithmic time complexity, making it much faster than both the **unsorted** and **sorted search** algorithms for larger arrays. As the table shows, **binary search** is also significantly faster than both **unsorted** and **sorted search**, particularly as the size of the array increases. The time complexity scaling of $O(\log(n))$ is evident in the relatively stable time across larger arrays.

The estimated time for a **binary search** in an array of 1 million elements would be approximately, $1000 - 2000ns$ ($1 - 2microseconds$). This time is significantly faster than the linear search method in a **sorted** and **unsorted array**. For 64 million elements, the number of comparisons would increase and using the assumption of 50-100 nanoseconds per comparison, the estimated search time for 64 million elements would be approximately, $1300 - 2600ns$ ($1.3 - 2.6microseconds$).

Recursive Binary Search

The fourth task involved implementing a **recursive** version of **binary search**. This approach used recursion to divide the search range in each call. The following code shows how the recursive function was implemented:

```
private static boolean recursive(int[] arr, int key, int min, int max) {
    int mid = min + ((max - min)/2);
    if (arr[mid] == key) {
        return true;
    }
    if ((arr[mid] > key) && (min < mid)) {
        return recursive(arr, key, min, mid - 1);
    }
    if ((arr[mid] < key) && (mid < max)) {
        return recursive(arr, key, mid + 1, max);
    }
    // as before
}
```

```
    return false;
}
```

The **recursive binary search** performs similarly to the iterative **binary search**, with execution times depending on the array size. The **recursive binary search** also has a time complexity of $O(\log(n))$. While recursive methods can simplify the code and enhance readability, they come with the overhead of repeated function calls, which can slow performance slightly compared to the iterative approach. Additionally, recursive calls carry the risk of **stack overflow** for very deep recursion levels. For moderate-sized arrays, **recursive binary search** is effective, but care should be taken with very large datasets due to potential stack space limitations. The number of **recursive calls** for different array sizes are as follows:

- For an array of length 1000: The number of recursive calls is $\log(1000) = 10$.
- For an array of length 2000: The number of recursive calls is $\log(2000) = 11$.
- For an array of length 4000: The number of recursive calls is $\log(4000) = 12$.
- For an array of length 1,000,000: The number of recursive calls is $\log(1000000) = 20$.

For most practical cases, the number of recursive calls in a binary search won't be problematic, even for large arrays (like a million elements). However, if you're working with extremely large datasets or systems with strict memory constraints, an iterative approach may be preferred to avoid potential **stack overflow** issues.

Cost of Sorting

The final task examined the **cost of sorting an array** before performing searches. Sorting is essential for **binary search** and can impact overall performance. This task aimed to evaluate the time complexity of sorting and its effect on search efficiency.

Sorting is beneficial when multiple searches are performed on the same dataset. However, the decision to sort depends on the frequency of searches versus the cost of sorting. For a single search, sorting might not be cost-effective, but for frequent searches, sorting can provide significant performance improvements.

Sorting is generally worth it if you perform many searches on the same dataset. For example, if you're performing multiple duplicate searches on an

array, sorting the data once may be worthwhile, as it allows you to use efficient algorithms like `binary search` or more straightforward comparisons of adjacent elements.

However, if you're only searching once, sorting first may not always be justified due to the initial overhead. The cost of sorting the array could outweigh the benefit of faster searching.

Conclusion

In this report, we analyzed and benchmarked four search algorithms: `unsorted search`, `sorted search`, `binary search`, and `recursive binary search`. Our results showed that `unsorted search` has a time complexity of $O(n)$ and scales linearly with data size, making it inefficient for large arrays. `Sorted search` offers a slight improvement by leveraging the sorted order of the array, but it still scales linearly.

In contrast, `binary search`, with its $O(\log(n))$ time complexity, showed a substantial improvement in performance, especially for large datasets. The `recursive binary search` algorithm performed similarly to the `iterative binary search`, but introduces the potential for higher memory usage due to recursive function calls.

Through this exploration, we learned the importance of leveraging data structure properties (e.g., `sorted order`) to optimize search performance. `Binary search`, when applicable, is a highly efficient method that dramatically reduces search times for large datasets. Understanding these trade-offs is crucial in selecting the right search algorithm based on the problem's constraints and dataset characteristics.